

Front-End Basics

What is a variable

A variable is a named container used to store a value.

For example:

```
let age = 20;
```

Here:

- `age` → the **name of the variable**
- `20` → the **value**
- `=` → means **assign this value to this variable**

Imagine you have a box with a label

AGE

on it.

You put the number

20

inside that box.

Later, you can replace what's inside:

```
age = 21;
```

JavaScript gives us different ways to create these variables. The three keywords you'll see are `var`, `let`, and `const`.

The most important question is:

Can I change the value later?

`let` — "This value may change"

Think of `let` as:

"I'm putting something in this box, but I might replace it later."

Real-life example:

```
let score = 50; score = 80;
```

A student's score can change, so `let` makes sense.

`const` — "I don't want to reassign this"

"I'm creating this variable, and I don't want to replace its value."

Real-life example:

```
const birthYear = 2000;
```

Your birth year doesn't change, so there's no reason to reassign it.

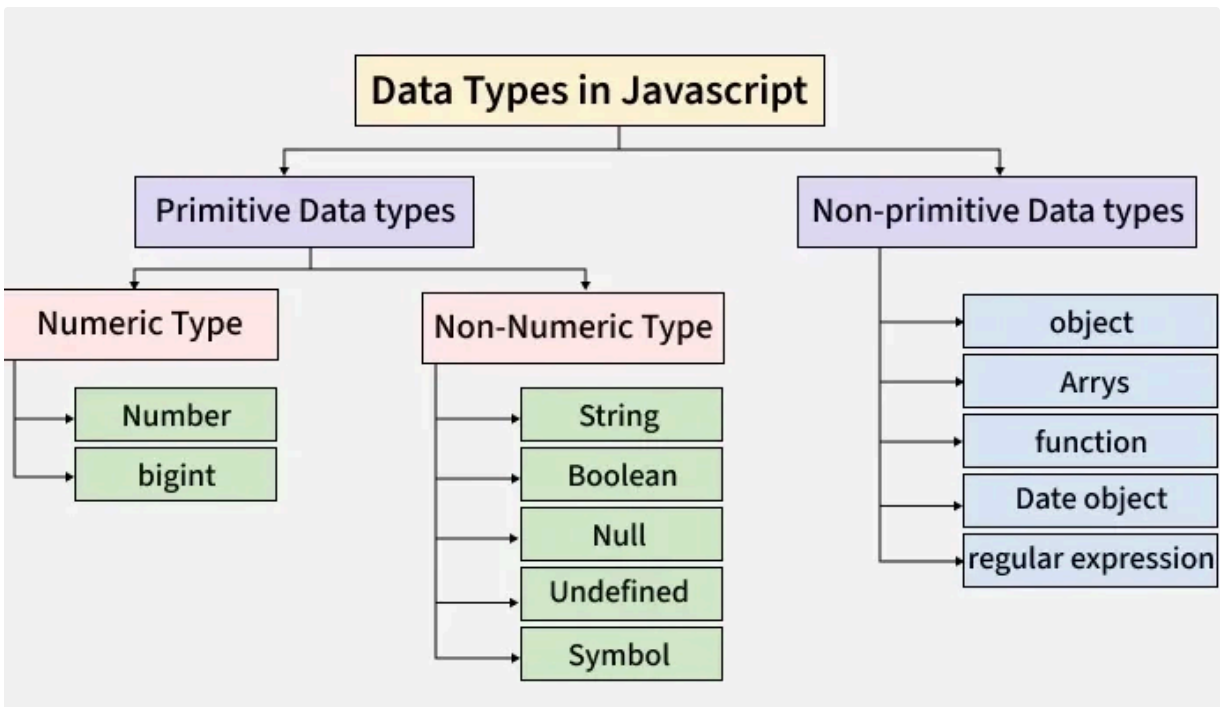
3. Then what is `var` ?

```
var age = 20; age = 21; //
```

Both `let` and `var` can be reassigned, but `var` is the older way of declaring variables. Modern JavaScript normally uses `let` and `const`.

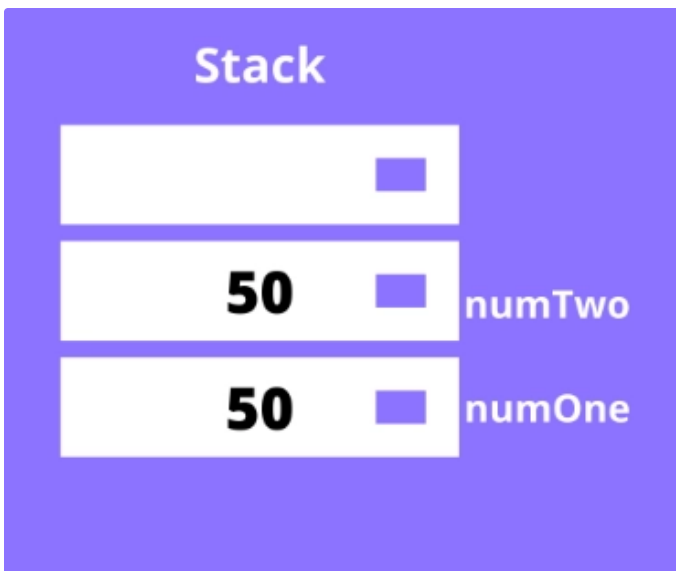
1. What is a Data Type?

A data type defines what kind of value a variable can store and what kind of operations can be performed on that value.



Primitive Data Types

Primitive data types are the basic, built-in data types in JavaScript that represent a single value. stored directly in memory, ensuring efficiency in both memory usage and performance.



The **stack** is a small, organized storage area where JavaScript keeps simple values associated with variables.

```
let numOne = 50; let numTwo = 50;
```

JavaScript creates a separate space for each variable and stores its value there.

String

Used for **text**.

```
let name = "Rahul"; let city = "Guwahati";
```

Number

Used for **numbers**, including integers and decimals.

```
let age = 20; let price = 99.50;
```

Boolean

Used for **true/false** values.

```
let isLoggedIn = true; let isStudent = false;
```

This becomes especially important when you teach conditions.

Undefined

A variable exists, but **has not been assigned a value**.

```
let age; console.log(age); // undefined
```

Null

Means **intentionally no value**.

```
let selectedUser = null;
```

`undefined` → don't have a value yet.

`null` → intentionally have no value.

BigInt

Used for **very large integers** beyond the safe range of the regular `Number` type.

```
let bigNumber = 123456789012345678901234567890n;
```

For beginners, just introduce the concept; they don't need to use it yet.

Symbol

Used to create **unique identifiers**.

a `Symbol` gives you: a unique identity.

```
const id1 = Symbol(); const id2 = Symbol(); console.log(id1 === id2); //false
```

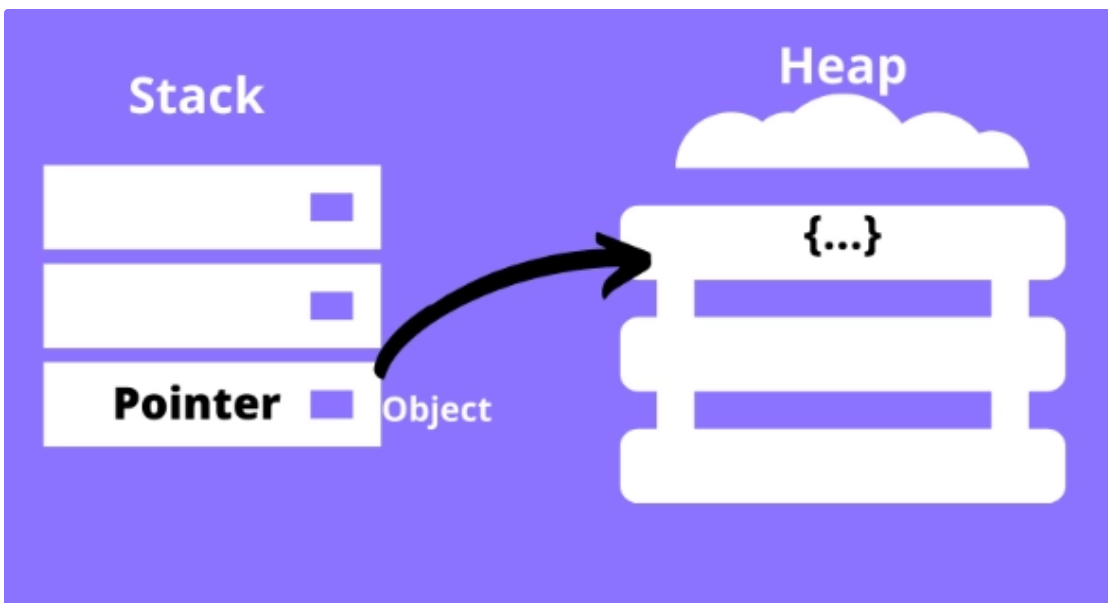
Because:

```
id1 → unique identity #1 id2 → unique identity #2
```

They're guaranteed to be different.

Non-Primitive Data Types

Non-primitive data types are used to represent more complex or structured data and can contain multiple values or properties. It is also known as derived data types or reference data types.



Objects → Heap + Reference

Now suppose:

```
let student = { name: "Rahul", age: 20 };
```

An object can be more complex and can contain multiple pieces of data.

Think of the **heap** as a **larger storage area** where JavaScript can keep these objects.

1. Object

An **object** stores related information using **key-value pairs**.

```
let student = { name: "Rahul", age: 20, course: "JavaScript" };
```

So instead of keeping three separate variables:

```
let name = "Rahul"; let age = 20; let course = "JavaScript";
```

we can group related information inside one object.

Simple definition for students:

Object = a collection of related data stored as key-value pairs.

2. Array

An **array** is used to store **multiple values in an ordered collection**.

```
let fruits = ["Apple", "Mango", "Banana"];
```

Think of it like a numbered list:

```
Index: 0 1 2 ↓ ↓ ↓ Apple Mango Banana
```

We can access individual values:

```
console.log(fruits[0]); // Apple console.log(fruits[1]); // Mango
```

Simple definition for students:

| Array = an ordered collection of multiple values stored in a single variable.

3. Function

A **function** is a reusable block of code that performs a particular task.

```
function greet() { console.log("Hello!"); }
```

Then we can call it:

```
greet();
```

Functions are technically **objects in JavaScript**, although beginners usually learn them as a separate concept because they behave differently and are fundamental to programming.

Simple definition:

Function = a reusable block of code designed to perform a specific task.

4. Date

The **Date** object is used to **work with dates and times**.

```
const today = new Date(); console.log(today);
```

5. RegExp — Regular Expression

A **RegExp** object is used to **search for, match, or validate patterns in text**.

For example:

```
const pattern = /hello/; console.log(pattern.test("hello world")); // true
```

Here **/hello/** is a pattern that looks for the word **"hello"**.

A practical example is checking whether text contains only numbers, whether an email follows a particular pattern, etc.

So:

RegExp is a built-in object used to work with patterns in text.

JavaScript Operators

Operators are symbols used to perform operations on values and variables.

1. JavaScript Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations such as addition, subtraction, multiplication, and division.

```
const sum = 5 + 3; // Addition
const diff = 10 - 2; // Subtraction
const product = 4 * 2; // Multiplication
const quotient = 8 / 2; // Division
console.log(sum, diff, product, quotient);
```

Common Arithmetic Operators

- **+** → Adds two values.
- **-** → Subtracts the second value from the first.
- ***** → Multiplies two values.
- **/** → Divides the first value by the second.
- **%** → Returns the remainder of a division.
- ****** → Raises a number to a power.

- `++` → Increases a value by 1.
- `--` → Decreases a value by 1.

Example:

```
console.log(10 % 3); // 1 console.log(2 ** 3); // 8
```

2. JavaScript Assignment Operators

Assignment operators are used to assign values to variables. They can also perform an operation and assign the result back to the variable.

```
let n = 10; n += 5; // n = n + 5 n *= 2; // n = n * 2 console.log(n); // 30
```

Common Assignment Operators

- `=` → Assigns a value to a variable.
- `+=` → Adds a value and assigns the result.
- `-=` → Subtracts a value and assigns the result.
- `=` → Multiplies a value and assigns the result.
- `/=` → Divides a value and assigns the result.
- `%=` → Takes the remainder and assigns the result.
- `*=` → Performs exponentiation and assigns the result.

For example:

```
let x = 10; x += 5; // x = 15 x *= 2; // x = 30 x -= 10; // x = 20
```

3. JavaScript Comparison Operators

Comparison operators are used to compare two values. They always return a **Boolean** value: `true` or `false`.

```
console.log(10 > 5); // true console.log(10 === "10"); // false
```

Common Comparison Operators

- `>` → Checks if the left value is greater than the right value.
- `<` → Checks if the left value is less than the right value.
- `>=` → Checks if the left value is greater than or equal to the right value.
- `<=` → Checks if the left value is less than or equal to the right value.
- `===` → Strict equality: checks whether both the **value and type** are the same.
- `!==` → Strict inequality: checks whether the **value or type** is different.
- `==` → Loose equality: compares values after type conversion.
- `!=` → Loose inequality: compares values after type conversion.

Example

```
console.log(10 > 5); // true console.log(10 < 5); // false console.log(10 =  
== 10); // true console.log(10 === "10"); // false console.log(10 !== "1  
0"); // true
```

Important: Prefer `===` and `!==` in modern JavaScript because they avoid unexpected type conversion.

4. JavaScript Logical Operators

Logical operators are used to combine or reverse conditions. They are commonly used with Boolean values (`true` and `false`) and in conditional statements.

```
const a = true; const b = false; console.log(a && b); // false console.log(a || b); // true console.log(!a); // false
```

Common Logical Operators

- `&&` → Logical AND. Returns `true` when **both operands are true**.
- `||` → Logical OR. Returns `true` when **at least one operand is true**.
- `!` → Logical NOT. Reverses the Boolean value.

Example:

```
const age = 20; const hasID = true; console.log(age >= 18 && hasID); // true
```

6. JavaScript Ternary Operator

The **ternary operator** is a shorthand way of writing a simple `if...else` condition.

It is called a ternary operator because it takes **three operands**:

```
condition ? expression1 : expression2
```

- If the condition is `true`, `expression1` is evaluated.
- If the condition is `false`, `expression2` is evaluated.

Example:

```
const age = 18; const status = age >= 18 ? "Adult" : "Minor"; console.log(status); // Adult
```

The same logic using `if...else` would be:

```
const age = 18; let status; if (age >= 18) { status = "Adult"; } else { status = "Minor"; } console.log(status);
```

So, the ternary operator provides a shorter way to write simple conditional logic.

7. JavaScript Comma Operator

The **comma operator** (`,`) evaluates multiple expressions from **left to right** and returns the value of the **last expression**.

Example:

```
let n1, n2; const result = (n1 = 1, n2 = 2, n1 + n2); console.log(result);  
// 3
```

stores `3` in `result`.

Important: The comma operator is different from the comma used to separate items in arrays, function arguments, or variable declarations. The comma operator specifically evaluates multiple expressions and returns the value of the last expression.

Simple Example

```
const result = (10, 20, 30); console.log(result); // 30
```

The expressions `10` and `20` are evaluated, but the comma operator returns the final value, `30`.

JavaScript Control Flow

Control flow is the order in which JavaScript executes statements in a program.

Normally, JavaScript executes code **from top to bottom**.

```
console.log("First"); console.log("Second"); console.log("Third");
```

Output:

```
First Second Third
```

But real programs need to make decisions and repeat tasks.

For example:

- If a student passes, display `"Pass"`.
- Otherwise, display `"Fail"`.
- Repeat something 10 times.
- Stop a loop when a condition is met.

This is where **control flow** comes in.

1. `if` Statement

The `if` statement is used to execute a block of code **only when a condition is true**.

Syntax

```
if (condition) { // code to execute }
```

Example

```
let age = 20; if (age >= 18) { console.log("You are an adult"); }
```

Here:

```
age >= 18
```

is the **condition**.

2. `if...else`

Sometimes we want to execute **one block** if the condition is true and **another block** if it is false.

For this, we use `if...else`.

Syntax

```
if (condition) { // code if condition is true } else { // code if condition is false }
```

Example

```
let age = 16; if (age >= 18) { console.log("Adult"); } else { console.log("Minor"); }
```

Output:

Minor

The important idea is:

`if` handles the true case, `else` handles the false case.

3. `else if`

What if there are **more than two possibilities**?

For example:

90-100 → A 80-89 → B 70-79 → C 60-69 → D Below 60 → F

We can use `else if`.

Syntax

```
if (condition1) { // code } else if (condition2) { // code } else if (condition3) { // code } else { // code }
```

Example

```
let marks = 85; if (marks >= 90) { console.log("Grade A"); } else if (marks >= 80) { console.log("Grade B"); } else if (marks >= 70) { console.log("Grade C"); } else if (marks >= 60) { console.log("Grade D"); } else { console.log("Grade F"); }
```

Output:

Grade B

Important

JavaScript checks the conditions **from top to bottom**.

As soon as it finds a condition that is **true**, its block executes and the remaining **else if** conditions are skipped.

For example:

```
let marks = 85; if (marks >= 90) { console.log("A"); } else if (marks >= 80) { console.log("B"); } else if (marks >= 70) { console.log("C"); }
```

JavaScript checks:

```
85 >= 90 → false 85 >= 80 → true
```

So it prints:

```
B
```

It does **not** continue checking the **70** condition.

4. Nested Conditions

A condition can exist **inside another condition**.

This is called a **nested condition**.

Example

```
let age = 20; let hasID = true; if (age >= 18) { if (hasID) { console.log("Entry allowed"); } }
```

Here:

First condition ↓ age >= 18 ↓ true ↓ Second condition ↓ hasID ↓ true ↓ Entry allowed

Nested **if...else**

```
let age = 20; let hasID = false; if (age >= 18) { if (hasID) { console.log("Entry allowed"); } else { console.log("ID required"); } } else { console.log("You must be 18 or older"); }
```

Output:

ID required

Important

Nested conditions are useful when the second decision **depends on the first decision**.

5. **switch** Statement

The **switch** statement is useful when we want to compare **one value against multiple possible values**.

Syntax

```
switch (value) { case value1: // code break; case value2: // code break; default: // code }
```

Example

```
let day = 2; switch (day) { case 1: console.log("Monday"); break; case 2: console.log("Tuesday"); break; case 3: console.log("Wednesday"); break; default: console.log("Invalid day"); }
```

Output:

Tuesday

Why do we use **break** ?

Consider:

```
let day = 2; switch (day) { case 1: console.log("Monday"); case 2: console.log("Tuesday"); case 3: console.log("Wednesday"); }
```

Output:

Tuesday Wednesday

Why?

Because after finding the matching case, JavaScript continues executing the following cases.

break tells JavaScript:

"Stop executing this switch statement."

Therefore:

```
case 2: console.log("Tuesday"); break;
```

stops the switch after Tuesday.

default

default executes when none of the cases match.

```
let day = 8; switch (day) { case 1: console.log("Monday"); break; case 2: console.log("Tuesday"); break; default: console.log("Invalid day"); }
```

Output:

Invalid day

if...else vs switch

if...else

Good when working with **conditions and ranges**:

```
if (marks >= 90) { } else if (marks >= 80) { }
```

switch

Good when checking **one value against specific possible values**:

```
switch (day) { case 1: break; case 2: break; }
```

6. for Loop

A loop allows us to **repeat code**.

Suppose we want to print:

```
1 2 3 4 5
```

We could write:

```
console.log(1); console.log(2); console.log(3); console.log(4); console.log(5);
```

But this becomes impractical when we need to repeat something hundreds or thousands of times.

That's why we use loops.

for Loop

The **for** loop is commonly used when we know **how many times** we want to repeat something.

Syntax

```
for (initialization; condition; update) { // code }
```

Example

```
for (let i = 1; i <= 5; i++) { console.log(i); }
```

Output:

```
1 2 3 4 5
```

Understanding the Three Parts

```
for (let i = 1; i <= 5; i++)
```

1. Initialization

```
let i = 1
```

Runs **once**, at the beginning.

It creates the loop variable.

2. Condition

```
i <= 5
```

Checked before every iteration.

If it is:

true → execute the loop false → stop the loop

3. Update

```
i++
```

Runs after each iteration.

It increases `i` by 1.

How the loop executes

```
i = 1 ↓ 1 <= 5 → true ↓ print 1 ↓ i++ ↓ i = 2 ↓ 2 <= 5 → true ↓ print 2 ↓  
... ↓ i = 6 ↓ 6 <= 5 → false ↓ STOP
```

for Loop Example

Print "Hello" five times:

```
for (let i = 1; i <= 5; i++) { console.log("Hello"); }
```

Reverse Loop

```
for (let i = 5; i >= 1; i--) { console.log(i); }
```

Output:

```
5 4 3 2 1
```

7. **while** Loop

A **while** loop repeats code as long as a condition is true.

Syntax

```
while (condition) { // code }
```

Example

```
let i = 1; while (i <= 5) { console.log(i); i++; }
```

Output:

```
1 2 3 4 5
```

How it works

Check condition ↓ true? / \ yes no ↓ ↓ execute STOP ↓ update ↓ check again

for vs while

Use a `for` loop when the number of iterations is generally known:

```
for (let i = 1; i <= 10; i++) { console.log(i); }
```

`while` Real-life example: Keep studying until the exam is understood

You don't know how many attempts it will take.

```
let understood = false; while (understood === false) { console.log("Study the topic..."); // eventually: understood = true; }
```

Think:

"I don't know how many times, so keep going while the condition is true."

8. `do...while` Loop

The `do...while` loop is similar to `while`, but there is one important difference:

The code inside **do** executes at least once.

Syntax

```
do { // code } while (condition);
```

Example

```
let i = 1; do { console.log(i); i++; } while (i <= 5);
```

Output:

```
1 2 3 4 5
```

The important difference

while

Condition is checked **first**.

```
let i = 10; while (i <= 5) { console.log(i); }
```

Output:

Nothing

Because:

```
10 <= 5 → false
```

do...while

Code executes **first**, then condition is checked.

```
let i = 10; do { console.log(i); } while (i <= 5);
```

Output:

```
10
```

Even though the condition is false.

Example - Suppose you want the user to enter a password.

They must get at least one chance to enter it.

```
let password; do { password = prompt("Enter your password:"); } while (password !== "1234"); console.log("Correct password!");
```

The first attempt happens **regardless of the condition.**

JavaScript enters the `do` block

```
do { password = prompt("Enter your password:"); }
```

The important thing about `do...while` is:

JavaScript does NOT check the condition first.

It simply says:

"Do this."

So the user gets asked:

Enter your password:

Suppose the user enters:

hello

Now:

```
password = "hello"
```

Step 3: NOW JavaScript checks the condition

```
while (password !== "1234");
```

JavaScript asks:

| "Is `password` NOT equal to `1234` ?"

We have:

```
password = "hello"
```

So:

```
"hello" !== "1234" ↓ true
```

Because the condition is `true`, we go back to the `do` block.

The user gets another chance:

```
Enter your password:
```

Step 4: User enters the correct password

Suppose they enter:

```
1234
```

Now:

```
password = "1234"
```

JavaScript checks:

```
password !== "1234"
```

Which becomes:

```
"1234" !== "1234" ↓ false
```

The condition is `false`.

So the loop stops.

Then JavaScript moves to:

```
console.log("Correct password!");
```

9. `break`

`break` is used to immediately terminate a loop.

Example

```
for (let i = 1; i <= 10; i++) { if (i === 5) { break; } console.log(i); }
```

Output:

```
1 2 3 4
```

When:

```
i === 5
```

becomes true, `break` terminates the loop.

Real-world Example

Suppose we are searching for a student:

```
const students = ["Rahul", "Priya", "Amit", "Neha"]; for (let i = 0; i < students.length; i++) { if (students[i] === "Amit") { console.log("Student found"); break; } }
```

Once Amit is found, there is no reason to continue searching.

10. `continue`

`continue` does **not** terminate the loop.

Instead:

`continue` skips the current iteration and moves to the next iteration.

Example

```
for (let i = 1; i <= 5; i++) { if (i === 3) { continue; } console.log(i); }
```

Output:

```
1 2 4 5
```

When `i` becomes `3`:

```
continue;
```

skips the rest of that iteration.

The loop continues with:

```
4
```

Suppose you're processing marks, but you don't want to process marks below 0 because they're invalid:

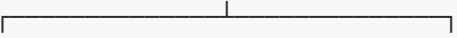
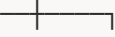
```
let marks = [85, 72, -1, 90, 65]; for (let i = 0; i < marks.length; i++) {
  if (marks[i] < 0) { continue; } console.log("Processing marks:", marks[i]);
}
```

Output:

```
Processing marks: 85 Processing marks: 72 Processing marks: 90 Processing m
arks: 65
```

Here, `-1` is skipped.

Control Flow Summary

CONTROL FLOW |  ↓ ↓ CONDITIONS LOOPS | | 
 |  ↓ ↓ ↓ ↓ ↓ ↓ if else switch for while do...wh
 ile | else if | nested if

Conditions

Statement	Purpose
<code>if</code>	Execute code when a condition is true
<code>else</code>	Execute code when the <code>if</code> condition is false
<code>else if</code>	Check additional conditions
Nested <code>if</code>	Put one condition inside another
<code>switch</code>	Compare one value against multiple cases

Loops

Loop	Purpose
<code>for</code>	Repeat code using initialization, condition, and update
<code>while</code>	Repeat while a condition is true
<code>do...while</code>	Execute at least once, then repeat while true

Loop Control

Keyword	Purpose
<code>break</code>	Immediately stop the loop
<code>continue</code>	Skip the current iteration

Function Declaration

A **function declaration** is the traditional way of creating a function.

Syntax

```
function functionName() { // code }
```

Example:

```
function greet() { console.log("Hello Students!"); }
```

This only **defines** the function.

It doesn't execute it yet.

To execute it, we **call** the function:

```
greet();
```

Parameters and Arguments

Functions can receive data from outside.

Parameter

A **parameter** is a variable defined inside the function's parentheses.

```
function greet(name) { console.log("Hello " + name); }
```

Here, `name` is a **parameter**. A parameter is a variable that receives a value when the function is called.

Argument

An **argument** is the actual value passed when calling the function.

```
greet("Rahul");
```

Here, `"Rahul"` is an **argument**.

So:

```
function greet(name) { console.log("Hello " + name); } greet("Rahul"); greet("Priya"); greet("Amit");
```

Output:

```
Hello Rahul Hello Priya Hello Amit
```

Remember

Parameter = placeholder

| Argument = actual value

Multiple Parameters

```
function add(a, b) { console.log(a + b); } add(10, 20);
```

Here:

- `a` , `b` → parameters
- `10` , `20` → arguments

Another example:

```
function introduce(name, age, course) { console.log("Name:", name); console.log("Age:", age); console.log("Course:", course); } introduce("Rahul", 20, "CSE");
```

Output:

```
Name: Rahul Age: 20 Course: CSE
```

Return Values

A function can send a result back using `return`.

```
function add(a, b) { return a + b; } let result = add(10, 20); console.log(result);
```

Output:

30

`console.log()` vs `return`

`console.log()`

→ Displays a value.

`return`

→ Sends a value back from the function.

Example:

```
function add(a, b) { return a + b; }
```

Now the returned value can be stored or used elsewhere:

```
let result = add(5, 3); console.log(result);
```

Important

When JavaScript reaches `return`, the function stops executing.

Simple example

```
function add(a, b) { console.log(a + b); } add(10, 20);
```

This shows **30** in the console, but the function does not give **30** back to us.

So this won't work:

```
let result = add(10, 20); console.log(result);
```

Output:

```
30 undefined
```

Why? Because `console.log()` only displays the value. It doesn't return the value from the function.

Now use **return**

```
function add(a, b) { return a + b; } let result = add(10, 20); console.log(result);
```

Output:

```
30
```

But now **result** actually contains **30**.

We can do more things with it:

```
let result = add(10, 20); console.log(result); console.log(result * 2); console.log(result + 100);
```

Output:

```
30 60 130
```

Function Scope in JavaScript

Function scope means that a variable declared with `var` inside a function is available **only inside that function**.

The easiest way to understand it is:

| A function creates its own "private area" for variables.

Global Scope

Variables declared **Globally** (outside any block or function) have **Global Scope**.

Global variables can be accessed from anywhere in a JavaScript program.

Variables declared with `var`, `let` and `const` are quite similar when declared outside a block.

They all have **Global Scope**:

```
var x = 1; // Global scope let y = 2; // Global scope const z = 3; // Global scope
```

Example

A variable declared outside a function, becomes **GLOBAL**.

```
let carName = "Volvo"; // code here can use carName function myFunction() {  
  // code here can also use carName }
```

Global variables has **Global Scope**:

All scripts and functions in the same web page can access a variable with global scope.

Function Scope

All JavaScript functions have their own scope.

Variables defined inside a function are not accessible (visible) from outside the function.

Variables declared with `var`, `let` and `const` are quite similar when declared inside a function.

They all have **Function Scope**:

```
function myFunction1() { var carName = "Volvo"; // Function Scope } function myFunction2() { let carName = "Volvo"; // Function Scope } function myFunction3() { const carName = "Volvo"; // Function Scope }
```

Example

Variables declared within a JavaScript function, are **LOCAL** to the function:

```
// code here can NOT use carName function myFunction() { let carName = "Volvo"; // code here CAN use carName } // code here can NOT use carName
```

Local Variables have Function Scope

- They can only be accessed from within the function
- No scripts or functions outside the function can access them
- Variables with the same name can be used outside the function
- Variables with the same name can be used in different functions
- Local variables are created when a function starts
- Local variables are deleted when the function is completed
- Arguments (parameters) work as local variables inside functions

Block Scope

Before ES6, JavaScript variables could only have **Global Scope** or **Function Scope**.

ES6 introduced two important new JavaScript keywords: `let` and `const`.

These two keywords provide **Block Scope** in JavaScript.

Variables declared with `let` and `const` inside a code block are "block-scoped," meaning they are only accessible within that block.

This helps prevent unintended variable overwrites and promotes better code organization:

Example

```
{ let x = 2; } // x can NOT be used here
```

Variables declared with the `var` keyword can NOT have block scope.

Variables declared with the `var` keyword, inside a `{ }` block, can be accessed from outside the block.

5. Function Expressions

Definition

A **function expression** is a function that is created and assigned to a variable.

Unlike a function declaration, the function is treated as a **value** and stored inside a variable.

Basic Syntax

```
const greet = function () { console.log("Hello!"); };
```

Here:

- `greet` → variable
- `function () { ... }` → function expression
- The function is assigned to `greet`

We call it using:

```
greet();
```

Function Expression with Parameters

Function expressions can accept parameters just like function declarations.

```
const add = function (a, b) { return a + b; }; console.log(add(10, 20));
```

Output:

```
30
```

Anonymous Function

A function expression often uses an **anonymous function**.

Anonymous means **without a function name**.

```
const greet = function () { console.log("Hello"); };
```

There is no name after `function` .

```
function () { ... } ↑ No function name → Anonymous Function
```

The variable `greet` is used to access the function.

Named Function Expression

A function expression can also have its own name.

```
const factorial = function fact(n) { if (n === 0) { return 1; } return n *  
fact(n - 1); }; console.log(factorial(5));
```

Output:

```
120
```

Here:

- `factorial` is the variable.
- `fact` is the function's internal name.

Named function expressions can be useful for things such as recursion and debugging.

USE CASES -

1. Function expressions can be assigned conditionally

This is one practical advantage.

```
let operation; if (true) { operation = function (a, b) { return a + b; }; }  
else { operation = function (a, b) { return a - b; }; } console.log(operation(10, 5)); // 15
```

Here, we are deciding **which function to store in the variable**.

That doesn't fit naturally with a simple function declaration.

2. Function expressions are useful for callbacks

You'll see this constantly in JavaScript.

```
setTimeout(function () { console.log("Hello after 2 seconds"); }, 2000);
```

The function doesn't need a separate name because we're simply giving it to

`setTimeout()` to execute later.

With an arrow function, this becomes:

```
setTimeout(() => { console.log("Hello after 2 seconds"); }, 2000);
```

This is one reason you'll see **function expressions** and **arrow functions** everywhere in modern JavaScript.

4. Function expressions give you more control over when the function is created/assigned

For example:

```
const isAdmin = true; const getMessage = isAdmin ? function () { return "Welcome Admin"; } : function () { return "Welcome User"; }; console.log(getMessage());
```

The variable `getMessage` receives one of two functions.

Arrow Functions

What is an Arrow Function?

An **arrow function** is another way of writing a function in JavaScript.

Instead of writing the `function` keyword, we use the arrow symbol:

```
=>
```

Normal Function

```
function greet() { console.log("Hello!"); }
```

Arrow Function

```
const greet = () => { console.log("Hello!"); };
```

Both create a function that prints `"Hello!"`.

So the simple definition is:

| Arrow function = a shorter and more concise way of writing a function.

The Basic Syntax

The general syntax of an arrow function is:

```
const functionName = (parameters) => { // function body };
```

For example:

```
const greet = (name) => { console.log("Hello " + name); };
```

Why do we need Arrow Functions?

You may be thinking:

| We already know functions. Why do we need another way to write them?

We can already write:

```
function add(a, b) { return a + b; }
```

There is nothing wrong with this.

But JavaScript uses **a lot of small functions**, especially when working with:

- arrays
- callbacks
- events
- asynchronous code
- higher-order functions

Writing the full `function` syntax every time can be unnecessarily long.

For example:

```
const numbers = [1, 2, 3, 4]; numbers.forEach(function(number) { console.log(number); });
```

The same thing using an arrow function:

```
numbers.forEach((number) => { console.log(number); });
```

So one major reason for arrow functions is:

They provide a shorter and cleaner syntax for many functions, especially small functions and callbacks.

There is another important reason that we will come to later:

Arrow functions behave differently from normal functions with `this`.

So arrow functions are not simply "shorter functions." They also have different behavior.

1. Function Declaration

```
function add(a, b) { return a + b; }
```

2. Function Expression

```
const add = function(a, b) { return a + b; };
```

3. Arrow Function

```
const add = (a, b) => { return a + b; };
```

All three can be used to create a function that adds two numbers.

The syntax is different.

When an arrow function has exactly **one parameter**, we may remove the parentheses:

```
const square = number => { return number * number; };
```

Both are valid:

```
const square = (number) => { return number * number; };
```

Basic Syntax

```
const functionName = (parameters) => { // function body };
```

Example:

```
const greet = (name) => { console.log("Hello " + name); };
```

Calling the function:

```
greet("Rahul");
```

Output:

```
Hello Rahul
```

7. Default Parameters

What are Default Parameters?

Normally, when a function has parameters, we expect the caller to provide the corresponding arguments.

For example:

```
function greet(name) { console.log("Hello " + name); } greet("Rahul");
```

Output:

```
Hello Rahul
```

Here: